Introduction to Digital Systems
Questions and Worked Solutions

Independent study edition

Editorial note

This independent English study edition reconstructs the handwritten-design questions from the supplied Academic Year 112 midterm and final examinations. Each question is followed by a worked solution and editable circuit drawings. Supplied answers are fallible references, not an authoritative answer key. Original question scans, photographs, and reference-answer files are not published.

Academic Year 112 corresponds to 2023–2024 in the local academic calendar; the supplied exam sheets do not print exact dates. The midterm's printed weights total 140%, and are preserved. The final is one 100% controller-design question. No simulation-software-only question was found in these two exam sheets. Unspecified conventions and additional reset assumptions are stated explicitly.

Contents

1 Midterm Examination: Academic Year 112 **1**

1. Simplification by Boolean identities (10%) . . . 1
2. Combining incompletely specified functions (10%) . . . 2
3. Five-variable minimum SOP (10%) . . . 2
4. A non-priority 7-to-3 encoder (10%) . . . 3
5. A two-bit 3-to-1 one-hot multiplexer (10%) . . . 4
6. Decoder outputs and maxterms (10%) . . . 4
7. A one-hot 1-to-4 demultiplexer (10%) . . . 5
8. Five-bit negative two's-complement numbers (10%) . . . 5
9. Division cells and a binary-to-BCD converter . . . 6
    (a) Add the quotient output (10%) . . . 6
    (b) Recover the decimal remainder with three full adders (10%) . . . 7
    (c) A complete 13-bit to four-digit BCD converter (20%) . . . 7
10. Signed two's-complement comparison (20%) . . . 9

Verification scope . . . 10

2 Final Examination: Academic Year 112 **11**

1. Automatic-door controller (100%) . . . 11

Solution: synchronous state machine . . . 12

State and output equations . . . 12

Gate-level state register and output decoder . . . 14

A five-second timer with no off-by-one error . . . 15

    Terminal-count logic . . . 15

Expanded counter circuit . . . 16

Enable gate and complete interconnection . . . 17

Verification and representative traces . . . 17

Chapter 1

Midterm Examination: Academic Year 112

The two supplied question pages contain Questions 1–10. The printed weights total 140%, including the three separately weighted parts of Question 9; those weights are retained. Every problem is a paper-and-pencil logic-design question. Throughout this chapter, juxtaposition means AND, $+$ or $\vee$ means OR, and an overbar means NOT. Binary indices use the leftmost listed variable as the most significant bit.

1. Simplification by Boolean identities (10%)

Question. Reduce F to a minimum sum-of-products expression. Use identities such as absorption and consensus; the stated hint is that the answer contains five literals.

$$\begin{aligned} F(a,b,c,d,e,f) &= (F_1 \vee \overline{F_2})(F_2 \vee F_3)F_2, \\ F_1 &= ac\bar{d} \vee bd \vee abc, \\ F_2 &= ac\bar{d} \vee bd \vee c\bar{e}f \vee d\bar{e}f \vee \bar{a}cef \vee \bar{a}ef, \\ F_3 &= (\bar{c} \vee e \vee \bar{f})(a \vee d \vee \bar{f})(a \vee \bar{e} \vee \bar{f}) \vee a\bar{c}f \\ &\quad \vee bd\bar{f} \vee aef \vee \bar{b}\bar{c}\bar{e} \vee \bar{c}\bar{e}f \vee \bar{c}d\bar{e}. \end{aligned}$$

Solution. Absorption removes the entire dependence on F_3:

$$(F_2 \vee F_3)F_2 = F_2, \qquad (F_1 \vee \overline{F_2})F_2 = F_1F_2.$$

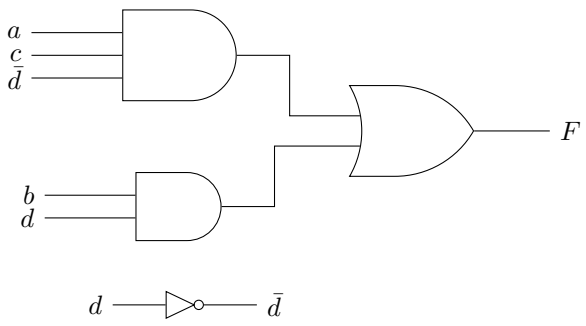
Let $G = ac\bar{d} \vee bd$. The term abc is the consensus of $ac\bar{d}$ and bd, since

$$abc = abc(d \vee \bar{d}) = abcd \vee abc\bar{d} \leq bd \vee ac\bar{d}.$$

Consequently $F_1 = G$. Also $F_2 = G \vee H$ for its remaining terms, and hence

$$\boxed{F = G(G \vee H) = ac\bar{d} \vee bd.}$$

The two displayed product terms contain $3+2=5$ literals. Both are needed: with $b=0$, the function is $ac\bar{d}$; with $a=0$, it is bd. A prime-implicant enumeration gives $ac\bar{d}$, bd, and the redundant consensus abc, and the first two are essential. Thus two terms and five literals are minimal. Variables e and f do not affect the result.



Signals bearing the same label are electrically connected. The supplied answer's final AND symbol between the two surviving products must be OR; their AND would be identically zero because it contains $d\bar{d}$.

2. Combining incompletely specified functions (10%)

Question. Let

$$S_1 = \{0, 3, 6, 11, 13, 15\}, \qquad S_2 = \{1, 2, 5, 14\},$$
$$S_3 = \{0, 5, 7, 9, 11, 15\}, \qquad S_4 = \{1, 3, 13, 14\}.$$

For $F(a, b, c, d) = [\Sigma m(S_1) \vee d(S_2)][\Sigma m(S_3) \vee d(S_4)]$, express F as an on-set plus a don't-care set, $\Sigma m(\cdot) \vee d(\cdot)$.

Solution. Here $d(S)$ denotes unspecified output values on S, not a Boolean signal that is automatically 1. In an AND, $0X = 0$, $1X = X$, and $XX = X$. The definite on-set and possible on-set are

$$M = S_1 \cap S_3 = \{0, 11, 15\}, \qquad P = (S_1 \cup S_2) \cap (S_3 \cup S_4).$$

The don't-care set is $P \setminus M = \{1, 3, 5, 13, 14\}$. Thus

$$\boxed{F = \Sigma m(0, 11, 15) \vee d(1, 3, 5, 13, 14).}$$

The complete three-valued calculation, with G_1 and G_2 denoting the two bracketed factors, is

Index	G_1	G_2	F	Index	G_1	G_2	F
0	1	1	1	8	0	0	0
1	X	X	X	9	0	1	0
2	X	0	0	10	0	0	0
3	1	X	X	11	1	1	1
4	0	0	0	12	0	0	0
5	X	1	X	13	1	X	X
6	1	0	0	14	X	X	X
7	0	1	0	15	1	1	1

The reference's on-set entry 1 is a transcription error for 11: index 1 is unspecified in both factors, while index 11 is definitely 1 in both. On-set and don't-care sets must be disjoint.

3. Five-variable minimum SOP (10%)

Question. Simplify $F(A, B, C, D, E) = \Sigma m(S_1) + d(S_2)$ to minimum sum-of-products form, where

$$S_1 = \{5, 13, 15, 19, 22, 23, 24, 26\},$$
$$S_2 = \{0, 3, 7, 8, 12, 16, 18, 21, 27, 30, 31\}.$$

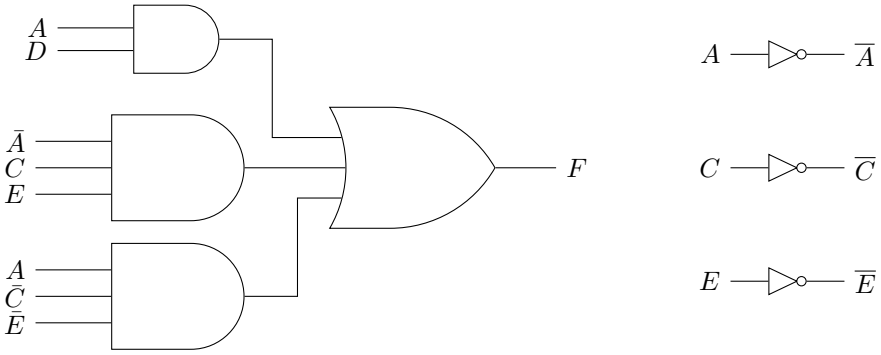
Solution. Combine on-set and don't-care minterms to form cubes, but require coverage only of the on-set. The eight prime implicants are shown below; a dash in an $ABCDE$ pattern means that the corresponding variable is omitted.

Pattern	Product	On-set minterms covered
`0110-`	$\bar{A}BC\bar{D}$	13
`0-1-1`	$\bar{A}CE$	5, 13, 15
`1-0-0`	$A\bar{C}\bar{E}$	24, 26
`1--1-`	AD	19, 22, 23, 26
`-01-1`	$\bar{B}CE$	5, 23
`-0-11`	$\bar{B}DE$	19, 23
`--000`	$\bar{C}\bar{D}\bar{E}$	24
`--111`	CDE	15, 23

Minterm 22 is covered only by AD, making that prime essential. After choosing it, minterms $5, 13, 15, 24$ remain. No one prime covers all four, so at least two more terms are required. The only three-literal prime that simultaneously covers $5, 13, 15$ is $\bar{A}CE$; minterm 24 can then be covered by either $A\bar{C}\bar{E}$ or $\bar{C}\bar{D}\bar{E}$. Two equally minimal answers are

$$\boxed{F = AD \vee \bar{A}CE \vee A\bar{C}\bar{E},} \qquad \boxed{F = AD \vee \bar{A}CE \vee \bar{C}\bar{D}\bar{E}.}$$

Both have three product terms and eight literals. There is no one-literal prime and AD is the only two-literal prime, so any three-term cover needs at least $2 + 3 + 3 = 8$ literals. This also proves the literal minimum. The accompanying Python program selects the second answer under its deterministic tie-break; the first is equally valid. They may differ on the don't-care inputs.



This drawing implements the first minimum. Repeated signal names denote connections; B is unused.

4. A non-priority 7-to-3 encoder (10%)

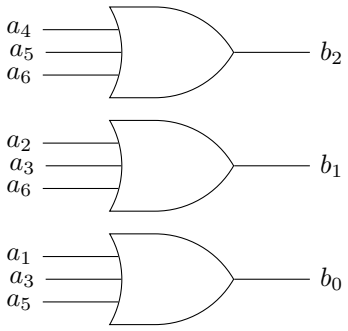
Question. Write the truth table and draw a gate-level implementation of a 7-to-3 encoder, not a priority encoder. If input a is all zero, output b must also be all zero.

Solution. The question does not number the seven input lines. Adopt the zero-based convention of the supplied reference: the inputs are $a[6:0]$, at most one is asserted, and an asserted $a[i]$ produces the binary index i. This convention maps both $a[0] = 1$ and the all-zero input to output zero; there is no separate valid flag.

$a[6:0]$	$b[2:0]$	$a[6:0]$	$b[2:0]$
`0000000`	`000`	`0001000`	`011`
`0000001`	`000`	`0010000`	`100`
`0000010`	`001`	`0100000`	`101`
`0000100`	`010`	`1000000`	`110`

Inputs with two or more asserted bits are outside this non-priority encoder's specified domain. The equations are

$$\boxed{b_2 = a_4 \vee a_5 \vee a_6, \quad b_1 = a_2 \vee a_3 \vee a_6, \quad b_0 = a_1 \vee a_3 \vee a_5.}$$



If inputs are instead named $a_1, \ldots, a_7$ and their numbers are to be encoded, the equally coherent one-based equations are $b_2 = a_4 \vee a_5 \vee a_6 \vee a_7$, $b_1 = a_2 \vee a_3 \vee a_6 \vee a_7$, and $b_0 = a_1 \vee a_3 \vee a_5 \vee a_7$. State the numbering convention; do not silently mix the two tables.

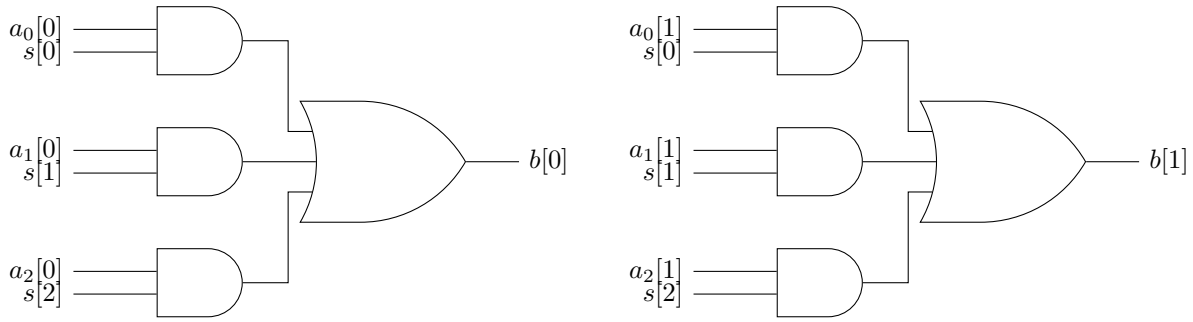
5. A two-bit 3-to-1 one-hot multiplexer (10%)

Question. Draw the gate-level implementation of a two-bit 3-to-1 multiplexer. Its inputs are $a_0[1:0]$, $a_1[1:0]$, and $a_2[1:0]$, selection is one-hot $s[2:0]$, and output is $b[1:0]$. Draw every bit individually; do not replace wires by a multi-bit bus notation.

Solution. For each output bit $j \in \{0, 1\}$,

$$b[j] = (s[0] \wedge a_0[j]) \vee (s[1] \wedge a_1[j]) \vee (s[2] \wedge a_2[j]).$$

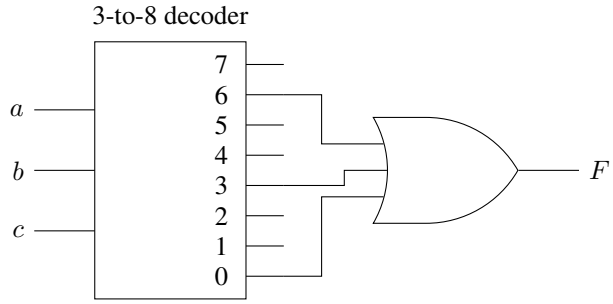
There are six two-input AND gates and two three-input OR gates. The following two networks show all six data bits and both output bits separately.



If the select vector is zero, this implementation returns zero. If multiple select bits are asserted, it bitwise-ORs those words; that behavior is outside the one-hot promise and is not priority selection. The reference drawing has four AND gates per output, which is not the requested three-input-word multiplexer.

6. Decoder outputs and maxterms (10%)

Question. The active-high outputs 6, 3, and 0 of a 3-to-8 decoder feed an OR gate. The decoder inputs are a (MSB), b, and c (LSB). Express $F(a, b, c)$ in product-of-maxterms notation $\Pi M(\cdot)$.



Solution. Each decoder output i is the minterm with binary index $i = 4a + 2b + c$. Therefore $F = \Sigma m(0, 3, 6)$, and its zeros are the five remaining indices:

$$\boxed{F = \Pi M(1, 2, 4, 5, 7).}$$

Expanding verifies the maxterm convention:

$$F = (a \vee b \vee \bar{c})(a \vee \bar{b} \vee c)(\bar{a} \vee b \vee c)$$
$$(\bar{a} \vee b \vee \bar{c})(\bar{a} \vee \bar{b} \vee \bar{c}).$$

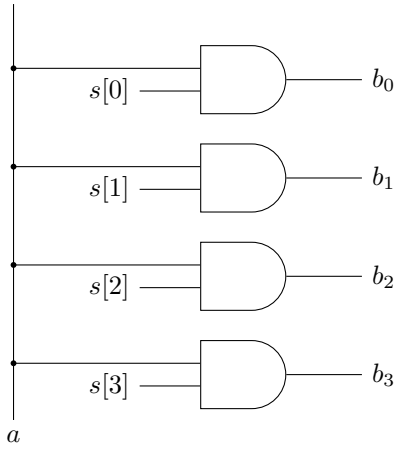
A maxterm vanishes at its named index, so a 1 in the index produces a complemented literal. In particular the last factor must vanish at 111, not 011 as in the supplied expansion.

7. A one-hot 1-to-4 demultiplexer (10%)

Question. A demultiplexer sends input a to one of b_0, b_1, b_2, b_3. Selection $s[3:0]$ is one-hot or all zero. If $s[i] = 1$, require $b_i = a$ and all other outputs zero. If s is all zero, all outputs must be zero. Draw a gate-level circuit.

Solution. Each output needs one two-input AND gate:

$$\boxed{b_i = a \wedge s[i], \qquad i = 0, 1, 2, 3.}$$



One-hot selection leaves one gate enabled; all-zero selection disables all four. Multiple asserted select bits are not part of the stated input domain.

8. Five-bit negative two's-complement numbers (10%)

Question. List every negative integer representable in five-bit signed two's-complement form, together with its bit pattern.

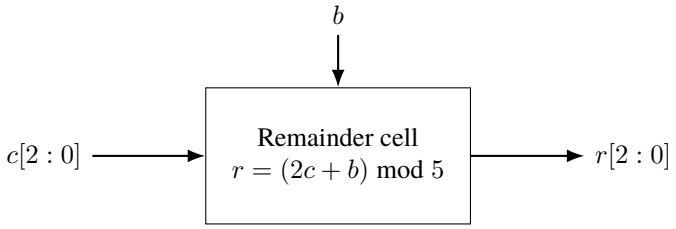
Solution. A five-bit signed word has range -16 through 15. A negative integer x is represented by the unsigned bit pattern for $32 + x$:

x	Bits	x	Bits	x	Bits	x	Bits
-1	11111	-5	11011	-9	10111	-13	10011
-2	11110	-6	11010	-10	10110	-14	10010
-3	11101	-7	11001	-11	10101	-15	10001
-4	11100	-8	11000	-12	10100	-16	10000

For example, -13 is $32 - 13 = 19 = 10011_2$. The most negative value has no positive five-bit counterpart; negating -16 in five bits overflows.

9. Division cells and a binary-to-BCD converter

Question. An existing combinational chain computes the remainder of unsigned $a[n-1:0]$ divided by 5. Each unit cell receives a three-bit old remainder $c[2:0]$ and the next input bit b, and produces a three-bit new remainder. The first cell receives $c_2 = 0$, $c_1 = a[n-1]$, $c_0 = a[n-2]$, and $b = a[n-3]$. Later cells consume the remaining input bits in descending order. The last remainder is a mod 5.



(a) Add the quotient output (10%)

Question. Add one output q to each unit cell so that the chain also produces the quotient $\lfloor a/5 \rfloor$. Design only this added output; the existing remainder outputs need not be redesigned.

Solution. An old remainder satisfies $0 \le c \le 4$. Appending a bit gives $x = 2c + b \in \{0, \ldots, 9\}$. The quotient bit and remainder must satisfy

$$x = 5q + r, \qquad 0 \le r < 5, \qquad q = \begin{cases} 0 & x < 5, \\ 1 & x \ge 5. \end{cases}$$

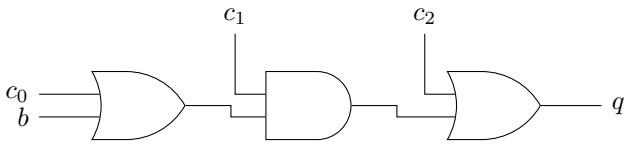
The complete valid-input table is

c	b	x	q	r	c	b	x	q	r
0	0	0	0	0	0	1	1	0	1
1	0	2	0	2	1	1	3	0	3
2	0	4	0	4	2	1	5	1	0
3	0	6	1	1	3	1	7	1	2
4	0	8	1	3	4	1	9	1	4

Since $c = 4$ asserts c_2, $c = 3$ asserts $c_1 c_0$, and $c = 2$ crosses the threshold only when $b = 1$,

$$\boxed{q = c_2 \vee c_1 c_0 \vee c_1 b = c_2 \vee c_1 (c_0 \vee b).}$$

The impossible old remainders 5, 6, and 7 are don't-cares, not extra operating states. A factored implementation is



The original answer's expression does not satisfy the table: at $c = 2, b = 0$ it incorrectly asserts q, and at $c = 4, b = 0$ it fails to assert q.

To see why the cascade yields the full quotient, suppose the processed prefix obeys $p = 5Q + c$. Appending b gives

$$2p + b = 5(2Q + q) + r.$$

Thus append each new cell's q bit to the accumulated quotient. The initial two-bit prefix is at most 3, so its quotient is zero and it can directly initialize c. There are $n - 2$ cells and $n - 2$ quotient bits $q[n - 3 : 0]$; the two leading full-width quotient bits are zero.

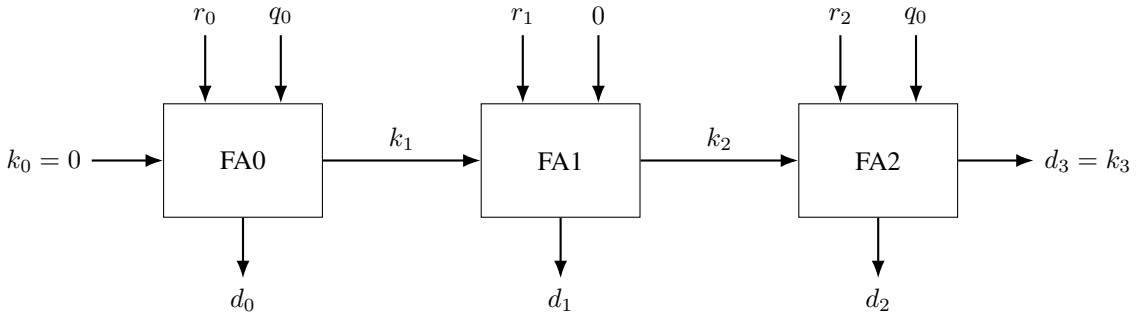
(b) Recover the decimal remainder with three full adders (10%)

Question. Given the final remainder $r_2r_1r_0 = a \bmod 5$ and the least significant quotient bit q_0, produce $a \bmod 10$ using exactly three full adders and no other logic.

Solution. Write $a = 5Q + r$ and $Q = 2T + q_0$. Then

$$a = 10T + 5q_0 + r, \qquad 0 \le 5q_0 + r \le 9, \qquad \boxed{d[3:0] = a \bmod 10 = r + 5q_0.}$$

The four-bit operand $5q_0$ is $(0, q_0, 0, q_0)_2$. Add it to $(0, r_2, r_1, r_0)_2$ using the three full adders below. The most significant output is the final carry; no fourth adder is needed.



In this diagram each full adder's top wires are its two operands, the left wire is carry-in, the right wire is carry-out, and the lower wire is the sum. Equivalently,

$$(d_0, k_1) = \mathrm{FA}(r_0, q_0, 0),$$
$$(d_1, k_2) = \mathrm{FA}(r_1, 0, k_1),$$
$$(d_2, d_3) = \mathrm{FA}(r_2, q_0, k_2).$$

Here FA returns $(\text{sum}, \text{carry})$. Only wires, fanout, the constant 0, and the three allowed full adders are used. Do not interchange r_0 and r_2: the reference figure's bit labels would reverse their binary weights.

(c) A complete 13-bit to four-digit BCD converter (20%)

Question. Construct a 13-bit binary-to-four-digit-BCD converter. The input is $a[12 : 0]$ and outputs are $b_3[3 : 0]$, $b_2[3 : 0]$, $b_1[3 : 0]$, and $b_0[3 : 0]$, with b_3 the most significant decimal digit. You may use the circuits from parts (a) and (b). Draw every unit cell explicitly; do not omit repeated cells for simplicity.

Solution. The key is that dividing an unsigned number by 10 can be performed by dividing by 5 and dropping the quotient's least significant bit:

$$\left\lfloor \frac{x}{10} \right\rfloor = \left\lfloor \frac{\lfloor x/5 \rfloor}{2} \right\rfloor .$$

Define $x^{(0)} = a$. At stage k, the division-by-5 chain produces quotient $q^{(k)}$ and remainder $R^{(k)}$. The three-full-adder circuit from (b), denoted D_k, gives

$$b_k = R^{(k)} + 5q_0^{(k)}, \qquad x^{(k+1)} = q^{(k)} \text{ with bit 0 discarded.}$$

The required widths and cell counts follow from the construction, including leading zeros where necessary:

Stage	Input width	Input maximum	Quotient bits	Next input	Cells
0	13	8191	$q^{(0)}[10:0]$	$x^{(1)}[9:0] = q^{(0)}[10:1]$	11
1	10	819	$q^{(1)}[7:0]$	$x^{(2)}[6:0] = q^{(1)}[7:1]$	8
2	7	81	$q^{(2)}[4:0]$	$x^{(3)}[3:0] = q^{(2)}[4:1]$	5

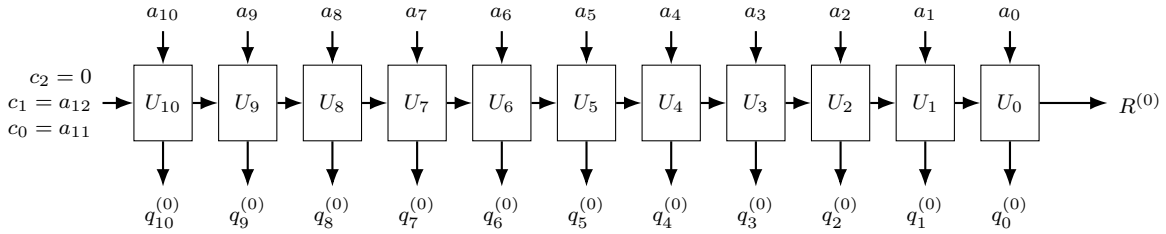
The final $x^{(3)} = \lfloor a/1000 \rfloor$ is between 0 and 8. Its four-bit binary encoding is already a valid BCD digit, so wire

$$\boxed{b_3[3:0] = x^{(3)}[3:0] = q^{(2)}[4:1].}$$

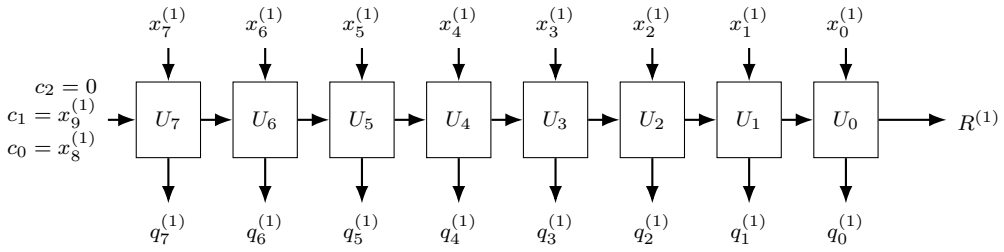
No fourth division stage is necessary. The design uses exactly $11 + 8 + 5 = 24$ unit cells and three copies of the three-full-adder digit circuit, hence nine full adders in those digit circuits.

Cell convention used in the complete drawings. Each box U_j is one existing remainder unit augmented with the quotient gate from (a). Its left and right wires carry three-bit c and r, its upper wire carries one input bit, and its lower wire carries one quotient bit. The three initial remainder bits are labelled individually beside the first cell. Every box and quotient tap is explicitly drawn below; there are no omitted cells. Each repeated signal label identifies the same wire in the following drawing or equation.

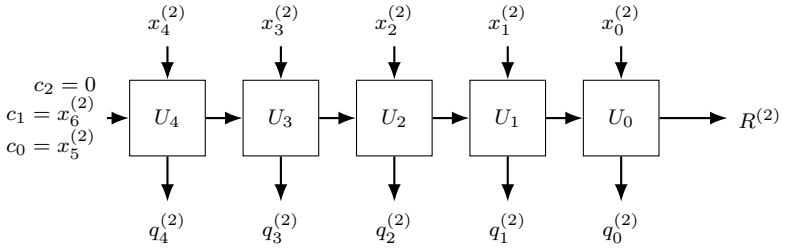
Stage 0: $x^{(0)} = a$, eleven cells.



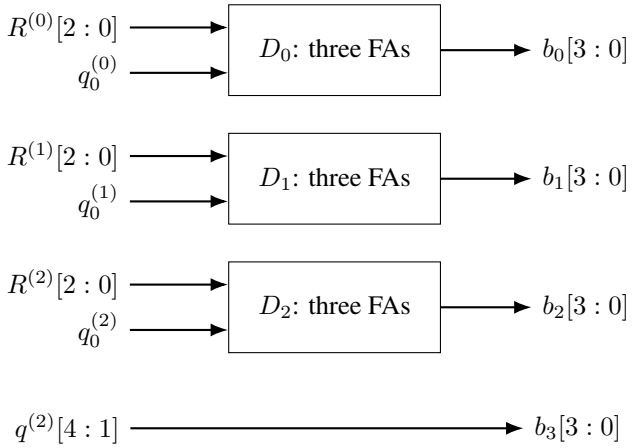
Stage 1: $x^{(1)}[9:0] = q^{(0)}[10:1]$, eight cells.



Stage 2: $x^{(2)}[6:0] = q^{(1)}[7:1]$**, five cells.**



All output connections. The digit circuits below are three distinct instances of the three-full-adder circuit already drawn in part (b). The direct fourth-digit connection is also shown.



Examples check both ends and a carry boundary: $0 \mapsto 0000$, $999 \mapsto 0999$, $1000 \mapsto 1000$, and $8191 \mapsto 8191$ in BCD digits. The published verification script simulates the unit-cell recurrence and the individual full adders for every input from 0 to 8191, and reconstructs the original integer from the four output digits.

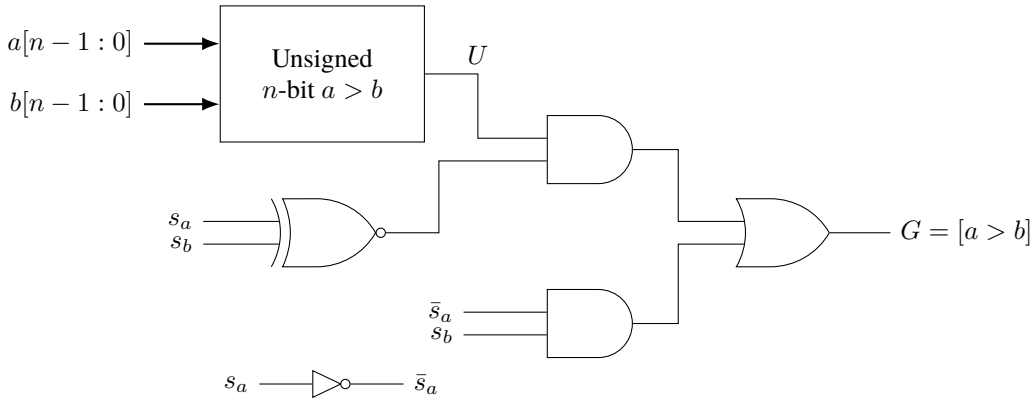
10. Signed two's-complement comparison (20%)

Question. Construct an n-bit magnitude comparator for signed two's-complement inputs $a[n-1:0]$ and $b[n-1:0]$. Its output must be 1 when $a > b$ and 0 otherwise. An unsigned magnitude comparator may be used.

Solution. Let $s_a = a[n-1]$ and $s_b = b[n-1]$ be the sign bits, and let U be the result of comparing the full n-bit patterns as unsigned integers. Opposite signs determine the answer immediately; with equal signs, unsigned and signed order agree because both values receive the same offset -2^n if negative.

$$\boxed{G = (\bar{s}_a s_b) \vee \left[\overline{s_a \oplus s_b} \wedge U\right], \qquad U = [a >_{\text{unsigned}} b].}$$

s_a	s_b	G	Reason
0	1	1	Every nonnegative value exceeds every negative value.
1	0	0	Every negative value is smaller than every nonnegative value.
0	0	U	Both are nonnegative.
1	1	U	Both signed values equal their unsigned values minus 2^n.



Equality makes $U = 0$, so the output is correctly zero. The construction also works for $n = 1$. It avoids a potentially overflowing signed subtraction. The supplied answer's prose labels the opposite-sign decisions as $a < b$ even though its truth decisions answer $a > b$; the required relation is $a > b$ throughout.

Verification scope

The companion script independently checks all 64 assignments of Question 1, all partial values in Question 2, every valid cube and minimum prime cover for Question 3, all promised encoder/mux/demux inputs, the complete decoder and five-bit negative-number tables, division cells, all 8192 BCD inputs, and all signed comparisons for widths 1–8. These are Boolean and arithmetic checks. They do not model propagation delay, hazards, or electrical implementation constraints not specified by the exam.

Chapter 2

Final Examination: Academic Year 112

1. Automatic-door controller (100%)

Question. Design a simple automatic-door controller with two inputs, D and Done, and two outputs, OP and CL.

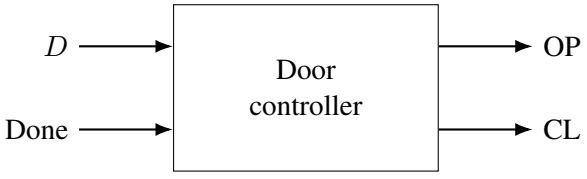
- D, detect: an infrared detector asserts this signal when a person is approaching the door.
- Done: the motor produces a one-clock-cycle pulse when the door reaches either the fully open or the fully closed endpoint.
- OP, open: command the motor to open the door.
- CL, close: command the motor to close the door.

The required behavior is:

1. While the door is closed and $D = 0$, keep it closed.
2. If a person is detected while it is closed, assert OP. Keep opening until Done $= 1$, then deassert OP.
3. While fully open, start closing after five continuous seconds with no person detected.
4. Any detection while fully open restarts the five-second interval.
5. If a person is detected while the door is closing, deassert CL, assert OP, and reopen until the fully open endpoint produces Done $= 1$.
6. Otherwise, keep closing until Done $= 1$, then deassert CL.

A 1 kHz clock is available: one cycle is 1 ms. An error of up to ± 0.2 s in the five-second delay is acceptable. Treat both inputs as synchronous to this clock.

Supply a state diagram, state equations, output equations, and a gate-level controller drawing. Explain the meaning of every state. Combinational components such as multiplexers and adders may be used as clearly labeled blocks. Flip-flops may be drawn without their internal circuit or clock wires, but their D and Q pins must be labeled. Sequential components, including counters and timers, must be expanded into their internal circuit. Hierarchical subcircuits are allowed when their connections are clear. The original instructions limit the examination answer to one sheet; this study edition expands the derivation and diagrams for readability.



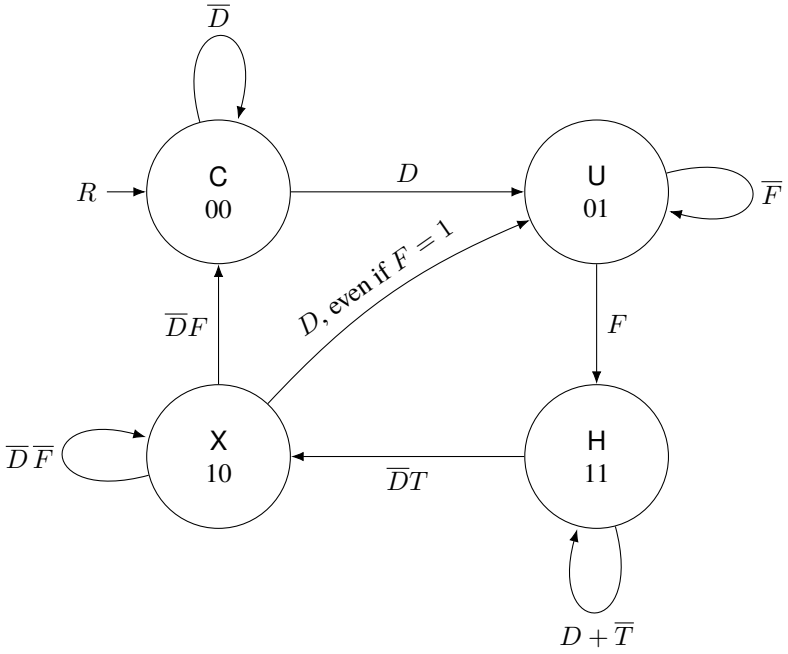
Solution: synchronous state machine

All input decisions below are made at a rising clock edge. Outputs are decoded from the state held after that edge. Thus detecting a person while closing switches the command to opening at the detecting edge, without an extra waiting state.

The question does not specify power-up initialization. For a reproducible implementation, assume the door is initially physically closed and apply a synchronous initialization signal R for one clock edge. It clears the state and timer registers. R is an implementation initialization condition, not an additional sensor or a third operational input in the question.

State	$s_1 s_0$	Meaning	(OP, CL)
C	00	Fully closed	$(0, 0)$
U	01	Opening	$(1, 0)$
H	11	Fully open; waiting	$(0, 0)$
X	10	Closing	$(0, 1)$

Write $F =$ Done. Let $T = 1$ mean that the timer contains 4999; its exact interpretation at the next clock edge is derived below. An unlabeled input in a transition condition is irrelevant.



The initialization arrow means that $R = 1$ forces C from every state; this override is omitted from individual arrows. In X, detection takes priority over an endpoint pulse: $D = F = 1$ requests reopening. In H, detection also takes priority over timeout and restarts the timer.

State and output equations

First write next-state functions before applying initialization. The high state bit is one for either a next state of H or a next state of X:

$$f_1 = s_1 s_0 + \overline{s_1} s_0 F + s_1 \overline{s_0}\,\overline{D}\,\overline{F}.$$

The low state bit is one for a next state of U or H. A direct state expansion simplifies to

$$\begin{aligned} f_0 &= \overline{s_1 s_0} D + \overline{s_1} s_0 + s_1 s_0 (D + \overline{T}) + s_1 \overline{s_0} D \\ &= D + s_0 (\overline{s_1} + \overline{T}). \end{aligned}$$

Therefore the D inputs of the two state flip-flops are

$$\boxed{s_1^+ = \overline{R} f_1, \qquad s_0^+ = \overline{R} f_0.}$$

The Moore outputs are

$$\boxed{\mathrm{OP} = \overline{s_1} s_0, \qquad \mathrm{CL} = s_1 \overline{s_0}.}$$

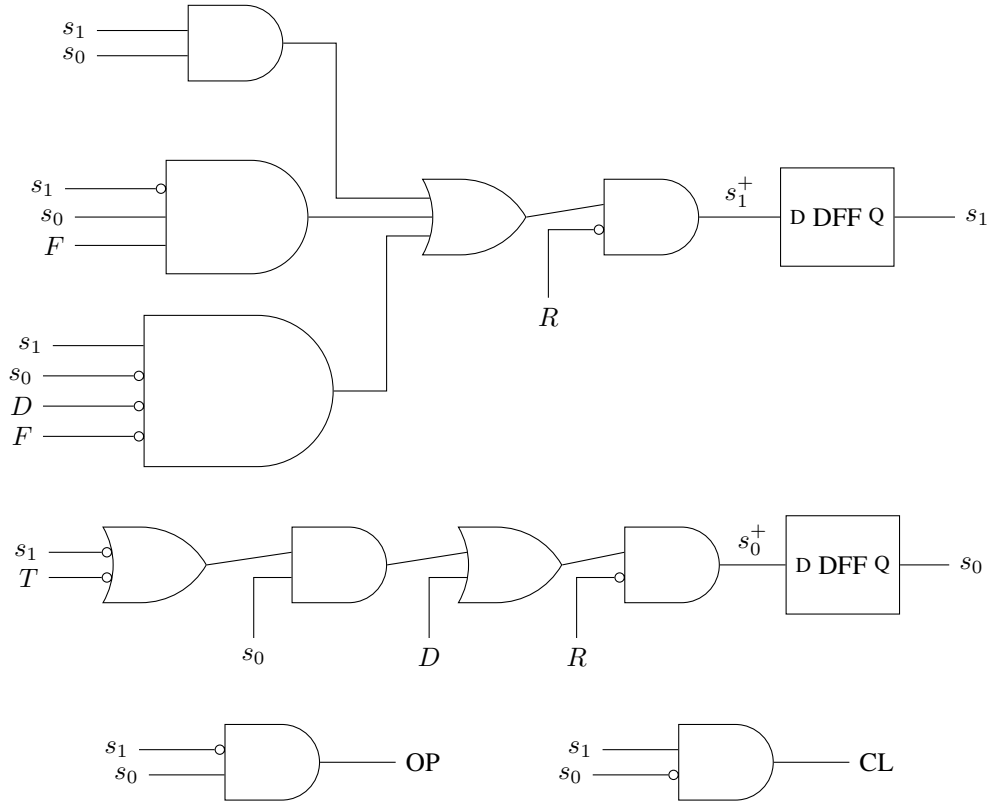
For every stable state, $\mathrm{OP}\,\mathrm{CL} = 0$. All four two-bit patterns are assigned states. The controller therefore needs only two state flip-flops, in addition to the timer flip-flops.

Current state	Condition	Next state	Timer action
C	$D = 0$ / $D = 1$	C / U	Clear
U	$F = 0$ / $F = 1$	U / H	Clear
H	$D = 1$	H	Clear
H	$D = 0, T = 0$	H	Increment
H	$D = 0, T = 1$	X	Clear
X	$D = 1$	U	Clear
X	$D = 0, F = 0$	X	Clear
X	$D = 0, F = 1$	C	Clear

In C, F is ignored; in U, D is ignored; in H, F is ignored. These are intentional consequences of the required operation, not missing cases.

Gate-level state register and output decoder

The following diagrams implement the equations directly. Identically named nets are connected wires. An input bubble denotes inversion; it is equivalent to an explicit NOT gate on that input. Each DFF symbol has its D pin on the left and Q pin on the right. All DFFs in this chapter use the same 1 kHz rising-edge clock; the clock wires are omitted as permitted by the question.



The two state-register circuits have no hidden storage. Their feedback comes from the labeled s_1, s_0 Q nets.

A five-second timer with no off-by-one error

A 13-bit unsigned register $c = c_{12} \cdots c_0$ is sufficient because $2^{12} < 5000 < 2^{13}$. During an uninterrupted open interval, count completed absent-person samples. Define

$$H = s_1 s_0, \qquad T = [c = 4999], \qquad E = \overline{R}\, H\, \overline{D}\, \overline{T}.$$

Here E enables incrementing. When $E = 0$, the counter is *cleared*, not held:

$$\boxed{c^+ = \begin{cases} c + 1, & E = 1, \\ 0, & E = 0. \end{cases}}$$

Because $H = 0$ before the edge that enters H, the timer is zero at that entry edge. The next 4999 absent samples produce the counts 1 through 4999. At the following, 5000th absent sample, $D = 0, T = 1$, so the state becomes X and the counter clears. The close command begins exactly $5000\,\text{ms} = 5\,\text{s}$ after the entry edge. A detection in H resets the same sequence.

Edge after restart	Pre-edge count	D	Post-edge state	Post-edge count
0	—	—	H	0
1	0	0	H	1
4999	4998	0	H	4999
5000	4999	0	X	0
Any edge in H	Any reachable count	1	H	0

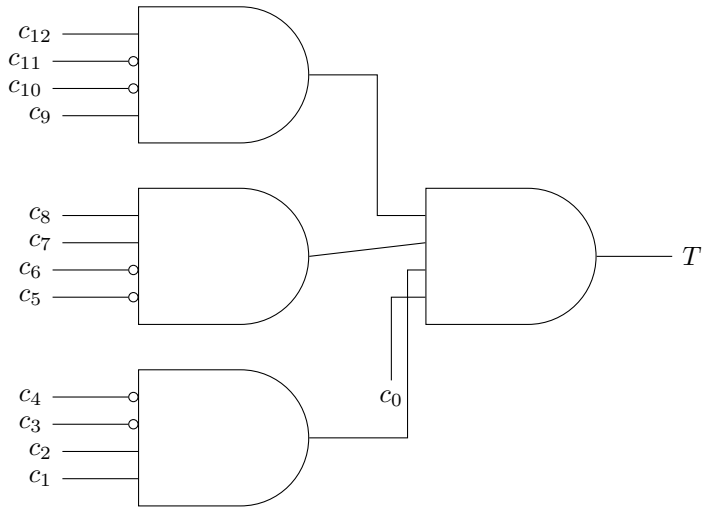
Thus $T = 1$ alone does not request closing: the transition also requires $D = 0$. A person arriving on the timeout edge keeps the door open. For asynchronous physical arrival times there would be sampling quantization; the exam explicitly supplies synchronous inputs.

Terminal-count logic

The required bit pattern is

$$4999 = 4096 + 512 + 256 + 128 + 4 + 2 + 1 = (1001110000111)_2.$$

The following four AND gates decode every bit. Input bubbles provide the six required complements.



Expanded counter circuit

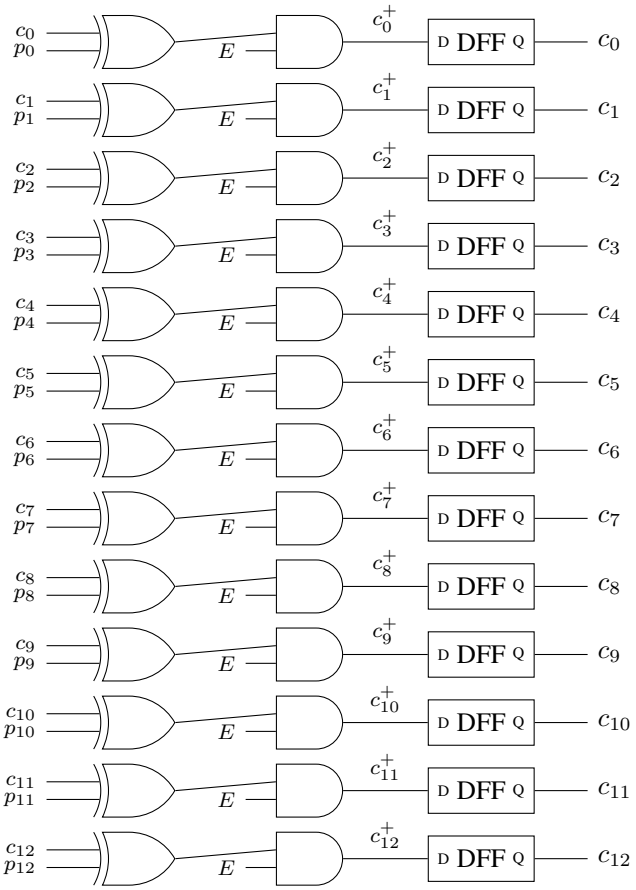
An incrementer has carry inputs

$$p_0 = 1, \qquad p_1 = c_0, \qquad p_2 = c_1 c_0, \quad \ldots, \quad p_{12} = c_{11} c_{10} \cdots c_0.$$

Equivalently $p_{i+1} = p_i c_i$ for $i = 0, \ldots, 11$. These are combinational carry signals, not registers. Thus every counter bit has the explicit next-state equation

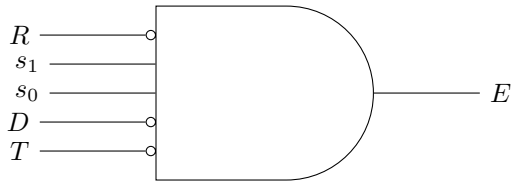
$$\boxed{c_i^+ = E(c_i \oplus p_i), \qquad i = 0, \ldots, 12.}$$

The AND with E implements the synchronous clear described above. The carry network may be built from the displayed AND recurrence or treated as the combinational part of an incrementer, as allowed by the question. All thirteen stored bits and their XOR/clear gates appear below; none is represented by an unexplained counter block.

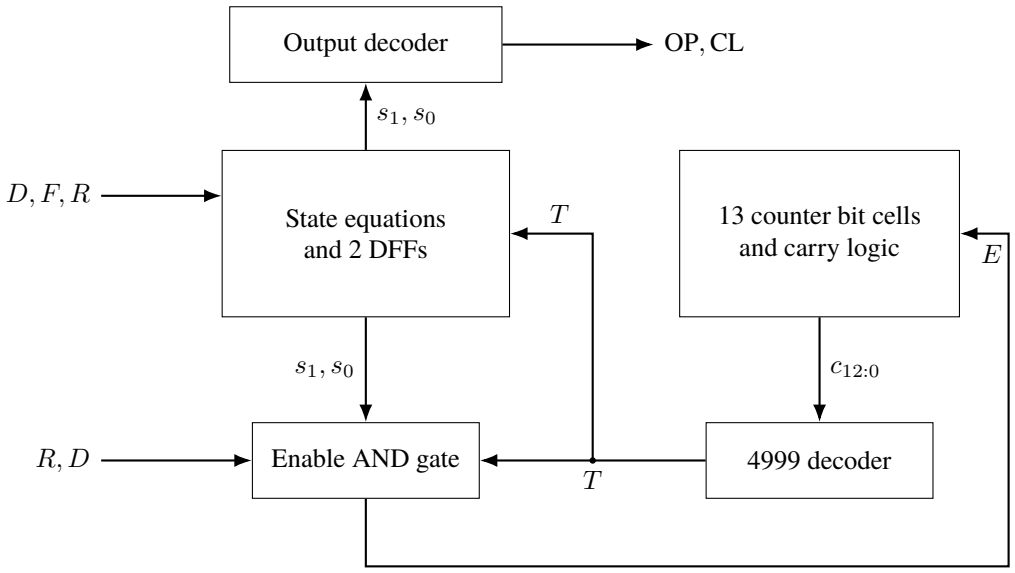


Every c_i net on a gate input connects to the corresponding Q output. All E nets share the same enable. The next section specifies its gate, and the labeled feedback connections complete the timer circuit.

Enable gate and complete interconnection



This implements $E = \overline{R}s_1s_0\overline{D}\,\overline{T}$. The complete interconnection below is a map of the already-expanded subcircuits, not a substitute for their flip-flop drawings.



The state logic receives D, F, T, R. The counter needs E and its own Q feedback; the decoder reads all thirteen Q bits. There are exactly 15 flip-flops: two for state and thirteen for time. No separate clock divider, latch, or hidden timer register is required.

Verification and representative traces

The accompanying `tools/verify_final.py` compares the Boolean next-state equations with a separately written transition-table model for every state and every combination of D, F, T, R. It also checks the XOR/AND counter circuit against arithmetic incrementing for every 13-bit pattern, and checks all reachable timer values with both input bits. It verifies the ideal synchronous examination circuit without requiring a logic-simulation application. Selected traces include:

1. Closed with no detection: remain closed, with zero commands and zero timer.
2. Detection, opening, endpoint pulse: enter H with timer zero.
3. 4999 absent samples: remain open. The 5000th absent sample starts closing.
4. Detection on the would-be timeout edge: remain open and clear the timer; require another full 5000 absent samples.
5. Detection while closing: enter U, with $(\text{OP}, \text{CL}) = (1, 0)$, including when $F = 1$ simultaneously.
6. Closing endpoint with $D = 0$: return to C.
7. Initialization from any state: return to C and clear all timer bits.